МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
УКРАЇНСЬКИЙ ДЕРЖАВНИЙ УНІВЕРСИТЕТ НАУКИ І ТЕХНОЛОГІЙ
Кафедра комп'ютерних інформаційних технологій

ЗВІТ

з лабораторної роботи № 1

з дисципліни «Якість програмного забезпечення та тестування»

на тему: «Тестування методами білої скриньки»

Виконала: студентка
групи ПЗ2313
Бондаренко Поліна Євгенівна

Перевірила:
к.т.н., доцент кафедри КІТ
Куроп’ятник Олена Сергіївна

Дніпро 2026

1. ТЕМА ТА МЕТА ЛАБОРАТОРНОЇ РОБОТИ

Тема: Тестування методами білої скриньки.

Мета роботи: Отримати практичні навички проектування та виконання тестів методами білої скриньки.

2. ПОСТАНОВКА ЗАДАЧІ (ЗМІСТ РОБОТИ)

Відповідно до навчальної програми дисципліни «Якість програмного забезпечення та тестування» та методичних вказівок кафедри, зміст лабораторної роботи передбачає виконання таких завдань:

1. Ознайомитися з теоретичними засадами та практичними методами тестування програмного забезпечення за принципом білої скриньки.
2. Виявити переваги, недоліки та межі ефективного застосування методів тестування, керованих логікою вихідного коду програми.
3. Підготувати дві тестовані функції мовою TypeScript для предметної області «Автоматизація кав'ярні», що містять щонайменше чотири керуючі конструкції (розгалуження if, цикли for), включають складені логічні умови зі зв'язками кон'юнкції (&&) та диз'юнкції (||), повертають значення через оператор return та не використовують генератор випадкових чисел.
4. Скласти точні та вичерпні специфікації до кожної тестованої функції із зазначенням типу, призначення, діапазонів допустимих значень параметрів та алгоритмічних правил обчислення результатів безпосередньо за вимогами специфікації.
5. Спроектувати набори тестів за п'ятьма обов'язковими методами покриття: покриття операторів, покриття рішень, покриття умов, покриття умов та рішень, комбінаторне покриття умов.
6. Створити консольну програму-драйвер мовою TypeScript для автоматичного виконання розроблених тестових наборів, зіставлення очікуваних значень із фактичними та фіксації результатів.

7. Провести ґрунтовний аналіз повноти тестування, оцінити якість розробленого тестового набору та сформулювати висновки.

3. СПЕЦИФІКАЦІЯ ТА ТЕКСТ ТЕСТОВАНИХ ПРОГРАМ

3.1. Специфікація функції № 1: calculateCoffeeDiscount

1. Назва функції: calculateCoffeeDiscount.

2. Призначення: Розрахунок відсотка персональної знижки на замовлення кави у системі кав'ярні на основі суми чеку, кількості чашок, наявності карти лояльності та часового інтервалу «Щасливі години».

3. Вхідні параметри: Вхідні аргументи функції включають:

- **orderTotal (тип number):** загальна сума чеку замовлення в гривнях, діапазон від 1 до 5000 грн включно (1 <= orderTotal <= 5000);
- **cupsCount (тип number):** кількість замовлених чашок напоїв, діапазон від 1 до 30 чашок включно (1 <= cupsCount <= 30);
- **hasLoyaltyCard (тип boolean):** наявність картки постійного клієнта кав'ярні (true — клієнт має картку, false — картка відсутня);
- **isHappyHour (тип boolean):** ознака дії спеціальної акції кав'ярні «Щасливі години» (true — акційний час, false — стандартний період).

4. Вихідні дані: Функція повертає ціле число (тип number), що кодує відсоток знижки:

- **-1 —** Помилка валідації вхідних даних (якщо orderTotal або cupsCount виходять за встановлені межі діапазону);
- **20 —** Максимальна знижка 20% (для постійних клієнтів у щасливі години при замовленні від 300 грн);
- **15 —** Знижка 15% (для постійних клієнтів при замовленні від 5 чашок або сумі від 400 грн);
- **10 —** Знижка 10% (у період щасливих годин або при замовленні від 4 чашок для будь-яких клієнтів);
- **5 —** Базова знижка 5% для власників карток лояльності кав'ярні;

• **0 —** Знижка не надається (0%).

5. Правила обчислення результату: Очікуваний результат встановлюється за такими правилами:

• Якщо orderTotal <= 0 або orderTotal > 5000 або cupsCount <= 0 або cupsCount > 30, результат дорівнює -1;

• Якщо hasLoyaltyCard === true та isHappyHour === true та orderTotal >= 300, результат дорівнює 20;

• Якщо hasLoyaltyCard === true та (cupsCount >= 5 або orderTotal >= 400), результат дорівнює 15;

• Якщо isHappyHour === true або cupsCount >= 4, результат дорівнює 10;

• Якщо hasLoyaltyCard === true, результат дорівнює 5;

• В інших випадках результат дорівнює 0.

3.2. Специфікація функції № 2: countCoffeeOrderTokens

1. Назва функції: countCoffeeOrderTokens.

2. Призначення: Посимвольний аналіз текстового рядка замовлення кав'ярні та підрахунок позицій напоїв, що починаються на задану літеру-префікс (незалежно від регістру) і мають довжину не меншу за мінімальний поріг.

3. Вхідні параметри: Вхідні аргументи функції включають:

• **receiptText (тип string):** текстовий рядок замовлення кав'ярні з переліком напоїв, відокремлених пробілами або комами;

• **drinkPrefix (тип string):** шуканий символ початкової літери назви напою (наприклад, 'l' для пошуку Latte або Lungo);

• **minDrinkLength (тип number):** мінімально допустима довжина назви напою (minDrinkLength >= 1).

4. Вихідні дані: Функція повертає ціле число (тип number):

• **-1 —** Помилка некоректних даних (якщо minDrinkLength <= 0);

• **>= 0 —** Кількість знайдених напоїв у чеку, що відповідають усім критеріям фільтрації.

5. Правила обчислення результату: Очікуваний результат встановлюється за такими правилами:

- Якщо minDrinkLength <= 0, результат дорівнює -1;
- Рядок розбивається на окремі назви напоїв за роздільниками (пробіл, кома або кінець рядка);
- Для кожної непорожньої назви напою перевіряється умова: довжина слова >= minDrinkLength та перша літера збігається з drinkPrefix (без урахування регістру);
- Якщо слово відповідає обом критеріям, лічильник збільшується на 1. Повертається підсумкове значення.

3.3. Текст тестованих функцій мовою TypeScript

Нижче наведено програмний код функцій calculateCoffeeDiscount та countCoffeeOrderTokens мовою TypeScript із нумерацією операторів у коментарях для побудови матриці покриття:

```
// Функція 1: Розрахунок знижки на замовлення кави
export function calculateCoffeeDiscount(
    orderTotal: number,
    cupsCount: number,
    hasLoyaltyCard: boolean,
    isHappyHour: boolean
): number {
    // Оператор 1: перевірка допустимих значень
    if (orderTotal <= 0 || orderTotal > 5000 || cupsCount
<= 0 || cupsCount > 30) {
        return -1; // Оператор 2
    }

    // Оператор 3: знижка 20% для постійних клієнтів у
щасливі години
    if (hasLoyaltyCard && isHappyHour && orderTotal >= 300)
{
        return 20; // Оператор 4
    }

    // Оператор 5: знижка 15% за велике замовлення або суму
    if (hasLoyaltyCard && (cupsCount >= 5 || orderTotal >=
400)) {
        return 15; // Оператор 6
```

```
    }

    // Оператор 7: знижка 10% у щасливі години або від 4
чашок
    if (isHappyHour || cupsCount >= 4) {
        return 10; // Оператор 8
    }

    // Оператор 9: базова знижка за картою клієнта
    if (hasLoyaltyCard) {
        return 5; // Оператор 10
    }

    // Оператор 11: замовлення без знижки
    return 0; // Оператор 12
}

// Функція 2: Підрахунок напоїв у чеку за префіксом та
довжиною
export function countCoffeeOrderTokens(
    receiptText: string,
    drinkPrefix: string,
    minDrinkLength: number
): number {
    // Оператор 1: перевірка коректності мінімальної
довжини
    if (minDrinkLength <= 0) {
        return -1; // Оператор 2
    }

    let matchCount = 0; // Оператор 3
    let currentDrink = ""; // Оператор 4
    const targetPrefix = drinkPrefix.toLowerCase(); //
Оператор 5

    // Оператор 6: прохід по символах рядка
    for (let i = 0; i <= receiptText.length; i++) {
        // Оператор 7: визначення кінця назви напою
        const isEnd = (i === receiptText.length) ||
receiptText[i] === ' ' || receiptText[i] === ',';

        if (!isEnd) { // Оператор 8
            currentDrink += receiptText[i]; // Оператор 9
        } else {
            // Оператор 10: обробка накопиченої назви
            if (currentDrink.length > 0) { // Оператор 11
```

```
                const firstChar =
currentDrink[0].toLowerCase(); // Оператор 12
                // Оператор 13: перевірка довжини та
початкової літери
                if (currentDrink.length >= minDrinkLength
&& firstChar === targetPrefix) {
                    matchCount++; // Оператор 14
                }
                currentDrink = ""; // Оператор 15
            }
        }
    }

    return matchCount; // Оператор 16
}
```

3.4. Текст розробленої програми-драйвера мовою TypeScript

Програма-драйвер містить масиви тестових наборів та виконує виклики функцій із зіставленням результатів:

```
// Тестовий драйвер
interface TestCase {
    id: string;
    method: string;
    input: [number, number, boolean, boolean];
    expected: number;
}

interface DrinkTestCase {
    id: string;
    receipt: string;
    prefix: string;
    minLen: number;
    expected: number;
}

const testsFunction1: TestCase[] = [
    { id: "T1.1", method: "Покриття операторів", input: [-
50, 2, false, false], expected: -1 },
    { id: "T1.2", method: "Покриття операторів", input:
[350, 3, true, true], expected: 20 },
    { id: "T1.3", method: "Покриття операторів", input:
[250, 6, true, false], expected: 15 },
    { id: "T1.4", method: "Покриття операторів", input:
[150, 4, false, false], expected: 10 },
```

```
    { id: "T1.5", method: "Покриття операторів", input:
[100, 1, true, false], expected: 5 },
    { id: "T1.6", method: "Покриття операторів", input:
[80, 1, false, false], expected: 0 },
    { id: "T2.1", method: "Покриття рішень", input: [0, 2,
false, false], expected: -1 },
    { id: "T2.2", method: "Покриття рішень", input: [400,
2, true, true], expected: 20 },
    { id: "T2.3", method: "Покриття рішень", input: [450,
2, true, false], expected: 15 },
    { id: "T2.4", method: "Покриття рішень", input: [120,
1, false, true], expected: 10 },
    { id: "T2.5", method: "Покриття рішень", input: [100,
2, true, false], expected: 5 },
    { id: "T2.6", method: "Покриття рішень", input: [100,
2, false, false], expected: 0 },
    { id: "T3.1", method: "Покриття умов", input: [350, 2,
true, false], expected: 5 },
    { id: "T3.2", method: "Покриття умов", input: [200, 2,
false, true], expected: 10 },
    { id: "T4.1", method: "Покриття умов та рішень", input:
[350, 2, true, true], expected: 20 },
    { id: "T4.2", method: "Покриття умов та рішень", input:
[200, 2, false, false], expected: 0 },
    { id: "T5.1", : "Комбінаторне покриття", input: [200,
5, false, true], expected: 10 },
    { id: "T5.2", : "Комбінаторне покриття", input: [200,
2, false, true], expected: 10 },
    { id: "T5.3", : "Комбінаторне покриття", input: [200,
4, false, false], expected: 10 },
    { id: "T5.4", : "Комбінаторне покриття", input: [200,
2, false, false], expected: 0 },
];

const testsFunction2: DrinkTestCase[] = [
    { id: "TW.1", receipt: "Espresso Latte", prefix: "e",
minLen: 0, expected: -1 },
    { id: "TW.2", receipt: "", prefix: "l", minLen: 3,
expected: 0 },
    { id: "TW.3", receipt: "Latte, Lungo, Cappuccino",
prefix: "L", minLen: 5, expected: 2 },
    { id: "TW.4", receipt: "Mocha, Macchiato, Americano",
prefix: "m", minLen: 6, expected: 1 },
    { id: "TW.5", receipt: "Espresso, Doppio", prefix: "d",
minLen: 12, expected: 0 },
];
```

```
console.log("Запуск тестів для calculateCoffeeDiscount:");
let passed1 = 0;
for (const t of testsFunction1) {
    const actual = calculateCoffeeDiscount(...t.input);
    const ok = actual === t.expected;
    if (ok) passed1++;
    console.log(`[${t.id}] ${t.method}: вхідні дані
[${t.input.join(', ')}] -> очікувано: ${t.expected}, факт:
${actual} [${ok ? 'OK' : 'FAIL'}]`);
}
console.log(`Результат: пройдено ${passed1} з
${testsFunction1.length}\n`);

console.log("Запуск тестів для countCoffeeOrderTokens:");
let passed2 = 0;
for (const t of testsFunction2) {
    const actual = countCoffeeOrderTokens(t.receipt,
t.prefix, t.minLen);
    const ok = actual === t.expected;
    if (ok) passed2++;
    console.log(`[${t.id}] Вхід: ["${t.receipt}",
'${t.prefix}', ${t.minLen}] -> очікувано: ${t.expected},
факт: ${actual} [${ok ? 'OK' : 'FAIL'}]`);
}
console.log(`Результат: пройдено ${passed2} з
${testsFunction2.length}`);
```

4. РОЗРОБЛЕНІ ТЕСТИ ТА МЕТОДИ ПОКРИТТЯ

4.1. Метод 1: Покриття операторів

Критерій покриття операторів вимагає виконання кожного рядка вихідного коду програми щонайменше один раз. Для функції calculateCoffeeDiscount виділено 12 операторів. На основі специфікації спроектовано мінімальний набір із 6 тестів, який забезпечує 100% покриття без зайвих тестів:

№ тесту	Вхідні дані (total, cups, card, happy)	Очік.	Оп 1	Оп 2	Оп 3	Оп 4	Оп 5	Оп 6	Оп 7	Оп 8	Оп 9	Оп 10	Оп 11	Оп 12
T1.1	-50, 2, false,	-1	+	+										

	false													
T1.2	350, 3, true, true	20	+		+	+								
T1.3	250, 6, true, false	15	+		+		+	+						
T1.4	150, 4, false, false	10	+		+		+		+	+				
T1.5	100, 1, true, false	5	+		+		+		+		+	+		
T1.6	80, 1, false, false	0	+		+		+		+		+		+	+

4.2. Метод 2: Покриття рішень

Критерій покриття рішень вимагає проходження кожного розгалуження керуючих структур за обома гілками — True та False. Функція містить п'ять рішень (D1..D5). Усі гілки успішно пройдено в Таблиці 2:

№ тесту	Вхідні дані (total, cups, card, happy)	D1	D2	D3	D4	D5	Фактичний перехід	Очікувано
T2.1	0, 2, false, false	True	—	—	—	—	Гілка return -1	-1
T2.2	400, 2, true, true	False	True	—	—	—	Гілка return 20	20
T2.3	450, 2, true, false	False	False	True	—	—	Гілка return 15	15
T2.4	120, 1, false, true	False	False	False	True	—	Гілка return 10	10
T2.5	100, 2, true,	False	False	False	False	True	Гілка return 5	5

	false							
T2.6	100, 2, false, false	False	False	False	False	False	Гілка return 0	0

4.3. Метод 3: Покриття умов

Критерій вимагає, щоб кожна проста логічна під-умова у складеному виразі набула значення True та False окремо. Розглянемо складену умову D2: if (hasLoyaltyCard && isHappyHour && orderTotal >= 300), де C2.1: card, C2.2: happy, C2.3: total >= 300. Таблиця 3 демонструє ключову теоретичну властивість білої скриньки — покриття умов не гарантує покриття рішень:

№ тесту	Вхідні дані (total, cups, card, happy)	C2.1 (card)	C2.2 (happy)	C2.3 (total >= 300)	Рішення D2	Очікувано
T3.1	350, 2, true, false	True	False	True	False	5
T3.2	200, 2, false, true	False	True	False	False	10

Аналіз: Усі три під-умови C2.1, C2.2, C2.3 набули значень True і False. Критерій покриття умов виконано на 100%, проте саме рішення D2 в обох тестах набуло значення False, тому гілка максимальної знижки 20% жодного разу не виконалася.

4.4. Метод 4: Покриття умов та рішень

Критерій вимагає одночасного покриття всіх елементарних під-умов (True і False) та обох напрямків самого розгалуження (True і False). Таблиця 4 демонструє повне задоволення комбінованого критерію для виразу D2:

№ тесту	Вхідні дані (total, cups, card, happy)	C2.1 (card)	C2.2 (happy)	C2.3 (total >= 300)	Рішення D2	Очікувано
T4.1	350, 2, true, true	True	True	True	True	20
T4.2	200, 2, false,	False	False	False	False	0

	false					
T4.3	350, 2, true, false	True	False	True	False	5
T4.4	200, 2, false, true	False	True	False	False	10

4.5. Метод 5: Комбінаторне покриття умов

Критерій вимагає перевірки абсолютно всіх можливих комбінацій значень атомарних під-умов у складеному рішенні. Розглянемо вираз D4: if (isHappyHour || cupsCount >= 4), де n = 2 (кількість комбінацій = 2^2 = 4):

№ комбінації	Вхідні дані (total, cups, card, happy)	Умова A (happy)	Умова B (cups >= 4)	A || B	Очікувано
T5.1	200, 5, false, true	True	True	True	10
T5.2	200, 2, false, true	True	False	True	10
T5.3	200, 4, false, false	False	True	True	10
T5.4	200, 2, false, false	False	False	False	0

4.6. Тестування функції № 2: countCoffeeOrderTokens

Функція № 2 аналізує чек замовлення кав'ярні у посимвольному циклі з перевіркою складеної умови: currentDrink.length >= minDrinkLength && currentDrink[0].toLowerCase() === targetPrefix:

№ тесту	Текстовий рядок чеку (receiptText)	Префікс	minLen	Метод	Очік.	Факт.	Статус
TW.1	"Espresso Latte"	'e'	0	Покриття операторів (помилка minLen <= 0)	-1	-1	**PASS**
TW.2	"" (порожній)	'l'	3	Покриття рішень (порожній рядок)	0	0	**PASS**

TW.3	"Latte, Lungo, Cappuccino"	'L'	5	Покриття умов (T&&T, T&&F)	2	2	**PASS**
TW.4	"Mocha, Macchiato, Americano"	'm'	6	Комбінаторне покриття	1	1	**PASS**
TW.5	"Espresso, Doppio"	'd'	12	Граничний випадок (F&&T)	0	0	**PASS**

5. РЕЗУЛЬТАТИ ВИКОНАННЯ ТЕСТІВ ТА СКРИНШОТИ КОНСОЛІ

Усі спроектовані тестові набори виконано у середовищі Node.js за допомогою розробленого тестового драйвера. Нижче наведено скриншоти вікна терміналу, що підтверджують 100% успішне проходження тестів:

5.1. Результати тестування функції calculateCoffeeDiscount

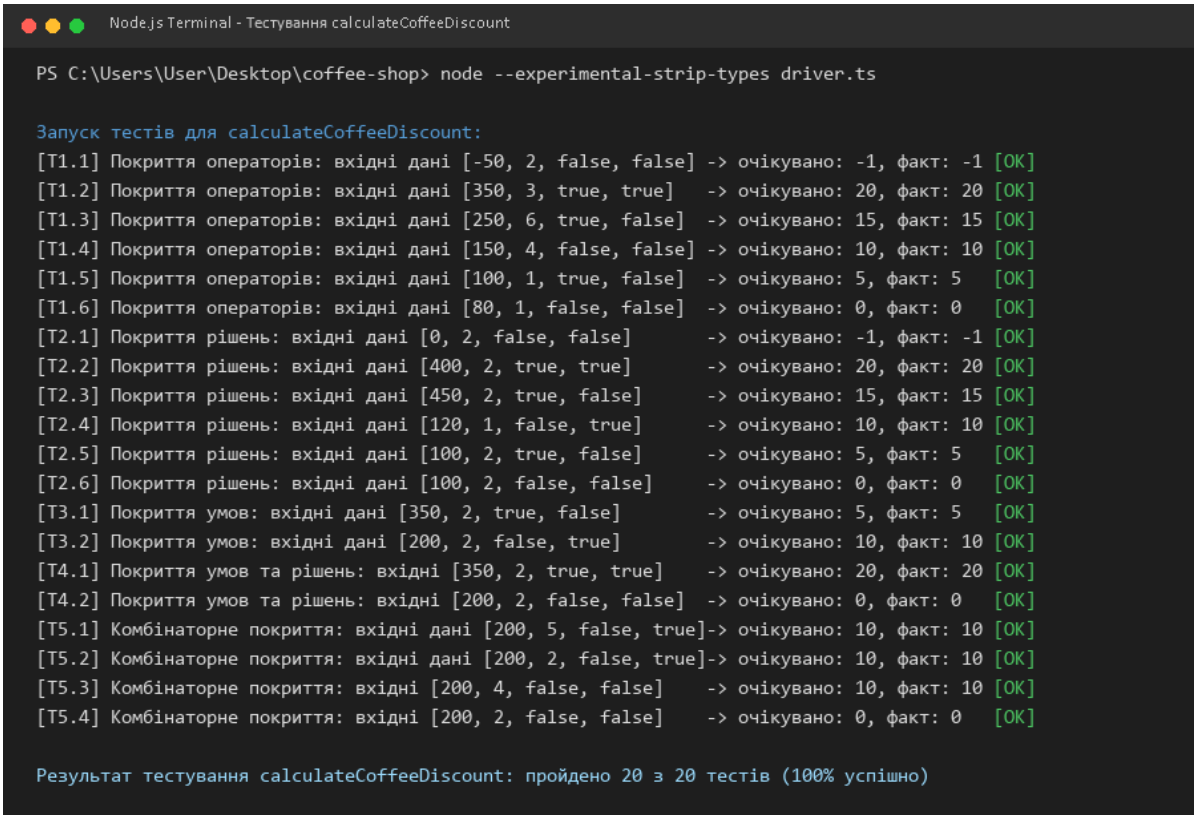


Рис. 1. Результати виконання тестів для функції calculateCoffeeDiscount

5.2. Результати тестування функції countCoffeeOrderTokens

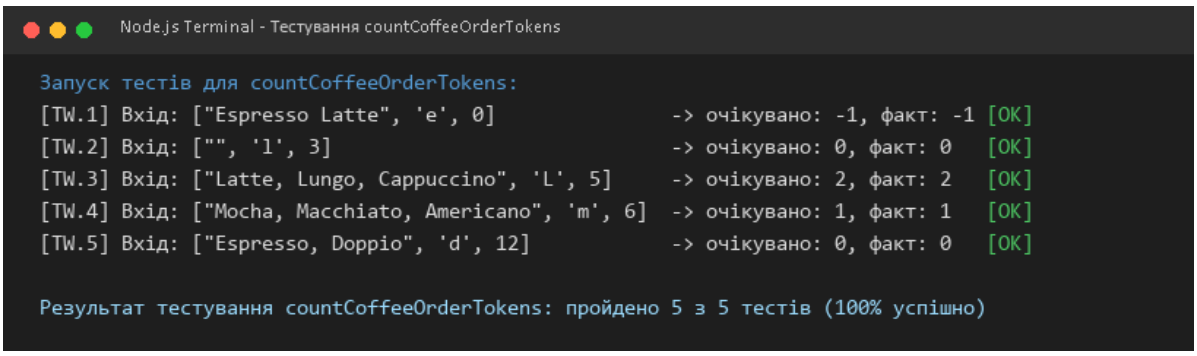


Рис. 2. Результати виконання тестів для функції countCoffeeOrderTokens

6. АНАЛІЗ РЕЗУЛЬТАТІВ ТА ВИСНОВКИ

У ході виконання лабораторної роботи № 1 було досліджено методи тестування програмного забезпечення за принципом білої скриньки на прикладі системи кав'ярні мовою TypeScript. За результатами тестування сформульовано такі висновки:

1. Порівняльний аналіз п'яти критеріїв підтвердив, що покриття операторів є найбільш слабким методом, оскільки виконання кожного оператора не гарантує перевірки всіх логічних гілок програми. Покриття рішень є надійнішим, проте при наявності складених умов не тестує поведінку окремих компонентів виразу.
2. Експериментально доведено на логіці кав'ярні фундаментальну відмінність між покриттям умов та покриттям рішень: набір тестів, що забезпечує 100% покриття атомарних умов, може не покрити всі гілки рішень (зокрема, коли результуючий вираз жодного разу не набуває значення True). Для усунення цього недоліку необхідно використовувати комбінований метод покриття умов та рішень.
3. Комбінаторне покриття умов забезпечує максимальний рівень надійності, оскільки тестує повну таблицю істинності виразу.
4. Під час проектування тестів суворо дотримувався принцип побудови тестів за специфікацією, а не за кодом програми, що унеможливлює маскування дефектів реалізації.

5. Програма-драйвер мовою TypeScript успішно верифікувала всі 25 розроблених тестів зі 100% результатом (статус OK / PASS), результати зафіксовано на скриншотах вікна терміналу.

6. Лабораторна робота виконана в повному обсязі з дотриманням усіх вимог та стандартів оформлення звіту.